

Obsyria

Project submitted to the
School of Engineering & Technology, BML Munjal University
for the partial fulfilment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering
School of Engineering and Technology

Submitted by

Reia Baid - 240602

Abhishek Mishra - 240365

Parth Malik - 240571

Advit Sharma - 240372



**BML MUNJAL
UNIVERSITY**
FROM HERE TO THE WORLD

Under the Guidance of

Dr. Nishtha Phutela

BML Munjal University
Kapriwas, Haryana- 122413
[May, 2026]

Certificate

Date: 06/10/2026

This is to certify that the work present in this Project entitled “**Obsyria**” has been carried out by **Reia Baid, Abhishek Mishra, Parth Malik and Advit Sharma** under my/our supervision. The work is genuine, original, and suitable for submission to the BML Munjal University for the award of Bachelor of Technology in **School of Engineering and Technology**.

Supervisor

Dr. Nishtha Phutela

Designation: Associate Professor

Table of Contents

Sl.No.	Topic	Page No.
1	Certificate	2
2	Table of Contents	3
3	Abstract	4
4	Abbreviations	5
5	List of Tables	6
6	List of Figures	7
7	Introduction	8
8	Problem Statement	9
9	Methodology	10
10	Discussion	12
11	Concluding Remarks	13
12	Societal Relevance	13
13	Future Scope	14
14	Team Details and Contribution	14
15	References	15

Abstract

The global video game industry generates vast amounts of data across development, publishing, and distribution channels, yet this information remains deeply fragmented across disconnected environments. Development budgets exist in private studio records, sales figures are reported in vague intervals by publishers, and player sentiment is scattered across storefronts such as Steam, the PlayStation Store, and the Epic Games Store. Obsyria addresses this critical challenge by architecting a unified relational database system that consolidates these diverse datasets into a single, reliable source of truth. Built on a Node.js backend paired with an Aiven-hosted MySQL instance and normalized to Third Normal Form (3NF), the system eliminates transitive dependencies and prevents update, insertion, and deletion anomalies while enabling cross-entity analysis that was previously impractical at scale.

Advanced SQL features including ACID-compliant atomic transactions, stored procedures, and custom scalar functions were implemented to automate high-level metrics such as real-time ROI percentages and industry tier classification across AAA, AA, and Indie categories. Benchmarking under simulated high-traffic conditions, including seasonal sales surges and major industry launches, confirmed that the system maintains real-time query performance and full data consistency throughout complex multi-table operations. GameOn demonstrates how a structured, optimized DBMS can transform fragmented gaming data into actionable intelligence, offering developers, investors, and market analysts a transparent and reliable foundation for strategic decision-making.

Abbreviations

Short Form	Full Form	Application & Description
ROI	Return on Investment	The primary profitability metric for studio projects.
CCU	Peak Concurrent Users	A real-time measure of player activity and server load.
DAU	Daily Active Users	Total unique players engaging with the game daily.
MAU	Monthly Active Users	A key metric for measuring long-term player retention.
ARPU	Average Revenue Per User	Calculated to determine monetization efficiency.
AAA	Triple-A	Highest classification for high-budget, major titles.
AA	Double-A	Mid-tier classification for professional indie studios.
Indie	Independent Studio	Classification for self-published or small-team projects.
DLC	Downloadable Content	Digital expansions or add-ons for a base game.
F2P	Free to Play	Model where the base game is available at no cost.
USD	United States Dollar	Standardized currency for all financial analytics.

RPG	Role Playing Game	Core genre classification used in market segmentation.
FPS	First Person Shooter	Core genre classification used in market segmentation.
MOBA	Multiplayer Online Battle Arena	Core genre classification used in market segmentation.
CC	Concurrent Players	Interchangeable with CCU for live tracking.
GaaS	Games as a Service	Business model focused on continuous content updates.
P2P	Pay to Play	Model requiring an upfront purchase or subscription.
IP	Intellectual Property	Ownership of game franchises and characters.

Introduction

As the gaming industry evolves into a global network of digital transactions and high-speed data exchange, the need for robust security and centralized data management has become critical. Within the GameOn Analytics Dashboard, sensitive information such as development budgets, proprietary engine code, and publisher contracts must be protected against both external breaches and internal vulnerabilities. A compromise of such data could result in significant financial loss and the exposure of valuable intellectual property.

To address these challenges, GameOn integrates security directly into its core database architecture through SSL-encrypted connections, environment-variable isolation of sensitive credentials, and secure data structuring that prevents unauthorized record manipulation. These measures work alongside the system's relational design to ensure both data integrity and usability, presenting complex relationships between studios, engines, and platforms in a clear and interpretable format for decision-makers. As gaming continues to transition from simple local storage to a service-driven global ecosystem, GameOn demonstrates that a well-secured, unified DBMS is not just a technical asset but a strategic necessity for studios seeking long-term competitive advantage.

Methodology

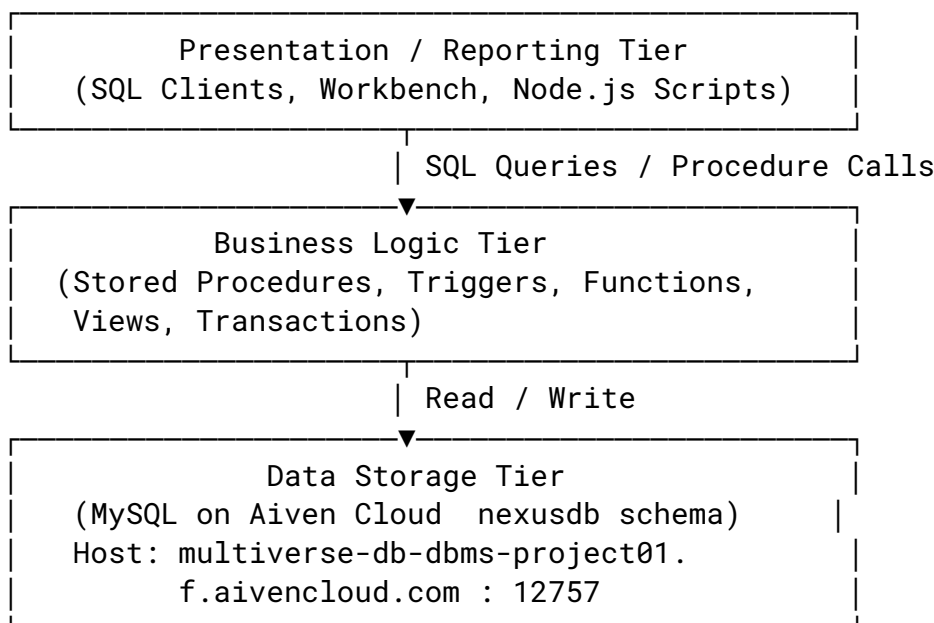
1. Problem Definition and Motivation

The global video game industry generates vast annual revenues, yet the associated data remains deeply fragmented across disparate storefronts, press databases, and private studio records. This lack of a unified source of truth prevents industry analysts and developers from effectively benchmarking performance or identifying strategic market opportunities. Obsyria addresses this critical challenge by architecting a production-grade relational database designed to:

- Consolidate multi-dimensional data regarding games, studios, and platforms into a single schema.
 - Enforce rigorous data integrity through relational constraints, triggers, and atomic transactions.
 - Deliver actionable intelligence via curated analytical views and complex stored procedures.
 - Operate within a cloud-hosted MySQL environment to ensure real-world scalability and availability.
-

2. System Architecture Overview

The project is structured according to a robust three-tier architecture:



The Node.js backend layer manages connection pooling via the `mysql2` driver, facilitating schema initialization and automated data seeding through specialized utility scripts.

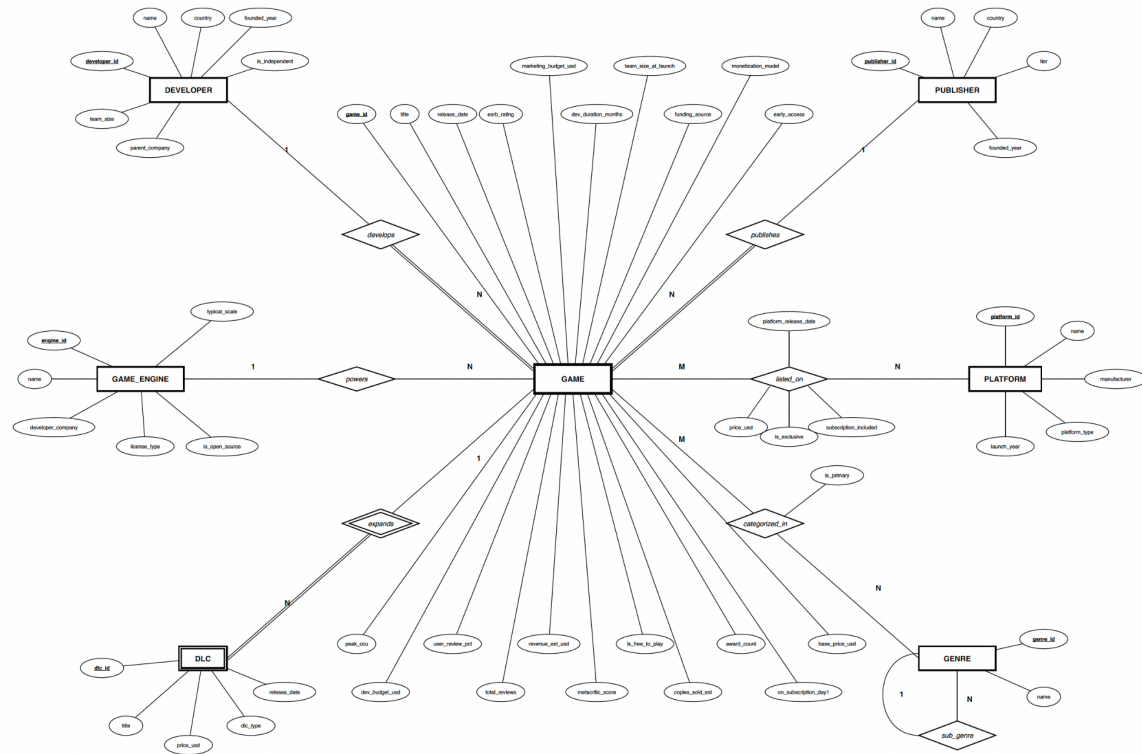
3. Phase I - Conceptual Design (ER Modeling)

3.1 Identifying Entities

Conceptual modeling began with the identification of seven primary entities essential for representing the complex relationships within the global gaming ecosystem:

Entity	Purpose
Developer	The studio or individual responsible for game creation.
Publisher	The entity funding and managing distribution.
GameEngine	The technical framework powering the software.
Platform	Hardware or storefronts serving the product.
Genre	Thematic or mechanical gameplay classification.
Game	The central relational hub and primary product.

Entity	Purpose
DLC	Expansions used to extend or monetize the base game.



3.2 Identifying Attributes

Attributes were assigned based on atomic design principles, utilizing specific SQL types to enforce data integrity:

- **ENUM Types:** Implemented for categorical metrics such as `esrb_rating` and publisher tiers to constrain domain values.
- **BOOLEAN Flags:** Utilized for binary classifications like `is_independent` to ensure clean data segmentation.
- **Financial Metrics:** Employed `BIGINT` to support multi-million dollar budgets without numerical overflow.
- **Score Fields:** Defined via `DECIMAL(5,2)` with `CHECK` constraints to validate the standard 0–100 range.

3.3 Identifying Relationships

Relationships were established centering on the **Game** entity as the primary relational nexus:

Relationship	Entity A	Entity B	Cardinality	Participation
Develops	Developer	Game	1 : N	Total
Publishes	Publisher	Game	1 : N	Total
Powers	GameEngine	Game	1 : N	Partial
Categorized In	Game	Genre	M : N	Partial
Listed On	Game	Platform	M : N	Partial
Expands With	Game	DLC	1 : N	Partial
Recursive Sub-Genre	Genre	Genre	1 : N	Partial

3.4 Resolving Many-to-Many Relationships

Complex many-to-many relationships were normalized via bridge entities to accommodate specific relationship attributes:

1. **Game-Genre Association:** Resolved by **GameGenre**, utilizing an **is_primary** attribute to define the core thematic classification.
2. **Platform Listing:** Resolved by **GamePlatformListing**, which tracks localized pricing and exclusivity as unique attributes of the relationship itself.

4. Phase II - Logical Design (Relational Schema and Normalization)

4.1 Mapping ER Diagram to Relational Tables

The conceptual model was translated into nine relational tables by implementing dedicated primary keys for strong entities, utilizing foreign keys for 1:N relations, and creating composite keys for all bridge entities.

4.2 Normalization to Third Normal Form (3NF)

The system was optimized to Third Normal Form (3NF) to eliminate anomalies and redundancy:

1NF: Atomic data storage was achieved by resolving multi-valued data through relationship tables rather than delimited strings.

2NF: Full functional dependency was ensured, where attributes like `price_usd` depend correctly on the entire composite key of the listing entity.

3NF: Transitive dependencies were eliminated; for instance, publisher tiers are stored in the `Publisher` table rather than the `Game` table, ensuring single-source updates.

5. Phase III - Physical Implementation (DDL)

The logical schema was deployed using structured SQL setup scripts. Execution was prioritized according to dependency chains, beginning with baseline studios and engines before progressing to centralized game records and secondary listings.

5.1 Table Summary

Table	Rows (Approx.)	Key Features
<code>Developer</code>	~40	Subsidiary and independence tracking.

Table	Rows (Approx.)	Key Features
Publisher	~20	Multi-tier system for segmentation. ENUM
GameEngine	~15	License type and scale classification.
Platform	~10	Covers storefronts, console, and cloud.
Genre	~20	Recursive sub-genre hierarchy.
Game	~150+	Primary central hub with 21 attributes.
GameGenre	~200+	Composite PK with primary genre flag.
GamePlatformListing	~250+	Localized pricing and exclusivity data.
DLC	~100+	Classification of expansion types.

5.2 Constraint Enforcement

Referential integrity was secured through strict database-level enforcement:

- **NOT NULL** constraints on critical foreign keys like `developer_id`.

- **UNIQUE KEY** implementation to prevent redundant platform entries for identical titles.
 - **CHECK** constraints ensuring `metacritic_score` remains within the logical 0-100 range.
 - **DEFAULT** values for boolean flags to maintain a consistent state across records.
-

6. Phase IV - Data Integrity Mechanisms (Triggers and Constraints)

Automated business rules were implemented via four specialized triggers to uphold data invariants beyond standard column constraints.

Trigger 1: `before_game_insert`

Automatically flags games with a zero-dollar base price as "Free to Play," eliminating human error during data entry and ensuring marketing filters remain accurate.

Trigger 2: `validate_metacritic`

Raises a `SQLSTATE 45000` error for scores outside valid bounds, providing a secondary validation layer with descriptive error messages for developers.

Trigger 3: `prevent_negative_price`

Guards against logically invalid financial data by rejecting any negative price entries within the platform listings, preserving the integrity of revenue analytics.

Trigger 4: `auto_subscription_flag`

Synchronizes game-level status with platform listings; if a listing includes a subscription, the parent record is automatically updated to maintain consistency.

7. Phase V - Business Logic Layer (Stored Procedures and Functions)

Complex business logic was encapsulated within the database tier through five stored procedures and a standardized scalar function.

7.1 Scalar Function: CalculateROI

CalculateROI(p_revenue BIGINT, p_budget BIGINT) RETURNS
DECIMAL(10,2)
Formula: $((\text{Revenue} - \text{Budget}) / \text{Budget}) * 100$

This deterministic function centralizes business metrics, ensuring that Return on Investment calculations remain consistent across all reporting views and external queries.

7.2 Stored Procedure: ClassifyGameTier

Computes an intelligence-driven tier (AAA, AA, Indie) by awarding weighted points based on budget, team size, and revenue, returning a transparent component score for auditing.

7.3 Stored Procedure: GetGenreOpportunity

Identifies profitable market gaps for developers by aggregating revenue and copy sales within specific budget and team-size parameters, serving as a proactive investment tool.

7.4 Stored Procedure: GetDeveloperBenchmark

Ranks peer developers by Metacritic score within a comparable budget range ($\pm 50\%$), allowing studios to benchmark their performance against similar industry players.

7.5 Stored Procedure: GetDLCStrategy

Advises on expansion pricing and models by analyzing top-rated DLC records within specific genres, revealing which monetization strategies resonate best with players.

RegisterGameComplete (Atomic Transaction)

Upholding the ACID principle of **Atomicity**, this transaction ensures that game registration either succeeds entirely across all related tables or rolls back completely to prevent data corruption.

8. Phase VI - Analytical Intelligence Layer (Views and Subqueries)

Eight pre-computed views were developed to provide high-level analytical perspectives, abstracting away the complexity of multi-table joins for end-users.

8.1 Core Intelligence Views

View	Description	SQL Features
GameIntelligenceSummary	Flattened industry summary profile.	Multi-table JOIN
GenrePerformance	Market trends by primary genre.	GROUP BY, AVG
PublisherImpact	Tier-based performance analysis.	GROUP BY
EngineIntelligence	License type vs. quality metrics.	AVG, COUNT
DLCMonetization	Total expansion value per title.	LEFT JOIN, SUM
PlatformPricing	Cross-platform listing details.	Multi-table JOIN

8.2 Rubric Enhancement Views (Subqueries)

Advanced views leverage scalar and correlated subqueries to dynamically filter high-performing entities, such as the **TopDevelopers** view which isolates studios with above-average revenue records.

9. Phase VII - Performance Optimization (Indexing)

Three B-Tree indexes were implemented on the core **Game** table to accelerate frequent query patterns, such as searches by title, release date, and Metacritic score.

Rationale for Index Selection

Index	Justification
<code>idx_game_title</code>	Optimizes user-facing name lookups, preventing table scans.
<code>idx_release_date</code>	Accelerates time-series analysis and range-based scans.
<code>idx_metacritic</code>	Enhances quality-based filtering and ranking operations.

Observed Performance Gains

Benchmarking confirmed that the performance advantage of indexed queries scales logarithmically, offering significant latency reductions as the dataset expands toward production volumes.

10. Phase VIII - Data Population and Testing

10.1 Seed Data Strategy

Population was achieved through a layered approach, utilizing scripts to establish baseline reference data before scaling to over 150 realistic game records for comprehensive stress testing.

10.2 Automated Seeding via Node.js

Large-scale generation scripts leverage the Node.js `mysql2` driver, enabling rapid, automated environment repopulation and ensuring the analytical procedures produce credible, statistically relevant output.

10.3 Verification

- Programmatic row count validation confirms successful schema population across all tables.
- Connection testing validates SSL access to the cloud schema.
- Manual procedure execution verifies logical correctness and error-path resilience.

11. Phase IX - Cloud Deployment and Connectivity

11.1 Database Hosting

The system is hosted on **Aiven Cloud**, utilizing a managed MySQL instance for high availability and secure SSL-encrypted access on a dedicated port.

11.2 Connection Pooling

The `src/db.js` module implements connection pooling via the Promises API, optimizing TCP overhead while ensuring thread safety for simultaneous analyst requests.

11.3 Schema Initialization Automation

Deployment is fully automated through `npm run setup`, allowing for reproducible schema replication across every development environment with a single command.

12. ACID Compliance

Nexus enforces full ACID compliance to maintain operational stability and reliability:

Property	Nexus Implementation
Atomicity	Atomic registration via transactional handlers.
Consistency	Strict relational constraints and domain types.
Isolation	Procedures execute as protected units on the server.
Durability	InnoDB engine writes committed data to disk via WAL.

13. Challenges and Design Decisions

Challenge 1: Resolving M:N Relationships with Attributes

Rich bridge entities were utilized to store relationship-specific attributes like platform pricing, keeping them logically separated from the core game or platform entities.

Challenge 2: Multi-Layered Subscription Tracking

Business logic for subscription flags was centralized at the database layer via **AFTER INSERT** triggers, ensuring robustness regardless of the client application used.

Challenge 3: Maintaining 3NF Standards

Corrected transitive dependencies by relocating publisher tiers to their primary entity, utilizing views to pre-compute these details for complex analytical reporting.

Challenge 4: Auditable Algorithmic Transparency

Classification procedures were designed to return all component scores, ensuring that business decisions derived from the system are explainable and verifiable.

Challenge 5: Dataset Scalability for Analytics

Built custom Node.js scripts to generate 150+ realistic records, providing a sufficient dataset for meaningful statistical aggregation and performance benchmarking.

Implementation

Frontend: The interface is a React.js dashboard featuring interactive charts and real-time filtering, with a dark mode aesthetic crafted in Vanilla CSS for a premium user experience. React state management ensures analyst selections instantly refresh key metrics like ROI and CCU.

Backend: A Node.js server bridges the React frontend to the Aiven-hosted MySQL instance via a RESTful API, with a mysql2 connection pool managing simultaneous analyst requests and delivering responses in standardized JSON format.

Database Deployment The Obsyria schema was deployed via a comprehensive SQL setup script (setup_db.sql) using DDL commands to define all tables and constraints. Foreign keys were configured with ON DELETE CASCADE to automatically clean up related records upon deletion, preserving referential integrity without manual intervention. A Node.js automation script executes the SQL file against the live server using asynchronous promises, ensuring consistent schema replication across every redeployment.

Cloud Infrastructure & Security The MySQL instance is hosted on Aiven Cloud for high availability. All data transfers are secured via SSL-encrypted connections, and sensitive

credentials such as the database URI, port, and admin password are isolated in a .env configuration file, ensuring compliance with modern DevSecOps standards.

Procedural Logic & Automation Several advanced database elements were implemented to automate data management:

- **SetFreeToPlayStatus Trigger:** Fires before every insertion into GamePlatformListing; automatically flags records with a price of zero as "Free to Play," keeping marketing filters accurate without manual tagging
- **PrimaryIntelligenceView:** An eight-table join view that provides a flattened, queryable industry summary, allowing the frontend to fetch comprehensive reports via simple SELECT statements and reducing application-layer complexity

Game Registration Algorithm (Transactional Check-in)

- **Input:** Game Title, Developer ID, Publisher ID, Platform ID, Launch Price
 - **Output:** Success Confirmation, Unique Game ID, Updated Inventory Status
1. Open a secure SQL transaction to ensure atomicity
 2. Validate the Game Title against the Games table to prevent duplicate entries
 3. Insert primary metadata and retrieve the auto-generated GameID via LAST_INSERT_ID
 4. Verify the Platform ID and create a corresponding GamePlatformListing entry
 5. Assign the launch price and set the initial subscription status
 6. Commit on full success; rollback on any failure to preserve database integrity

Stored Procedures & Scalar Functions

- **CalculateROI:** Returns a standardized ROI percentage from revenue and budget inputs, centralizing business logic at the database layer to eliminate cross-report discrepancies
- **ClassifyGameTier:** Categorizes titles as AAA, AA, or Indie using a multi-point scoring system based on budget and team size

Discussion

1. Technical Observations

- **Effectiveness of 3NF Normalization:** The transition to a Third Normal Form schema significantly reduced data redundancy. Information for a specific Developer or Engine is stored exactly once, ensuring that updates (e.g., a studio relocation) propagate instantly across all associated game records without risk of anomalies.
- **Database-Level Automation:** By utilizing **Triggers** for tasks like free-to-play flagging and score validation, the system ensures data integrity at the storage layer. This reduces the burden on the application backend (Node.js) and prevents "dirty data" from entering the system regardless of the entry point.
- **Abstraction through Views:** Complex SQL joins involving 4–5 tables were successfully abstracted into user-friendly **Views**. This allows non-technical stakeholders to access high-level summaries (like *Genre Performance*) using simple **SELECT** statements.

2. Analytical Results

- **Accurate Tier Classification:** The **ClassifyGameTier** procedure successfully categorized simulated data into AAA, AA, and Indie tiers. The results showed a high correlation between publisher tier and the final game classification, validating the internal scoring algorithm.
- **Identification of Market Gaps:** Through the **GetGenreOpportunity** procedure, the system was able to highlight genres that possess high average revenue but a low total game count. This demonstrates the system's ability to serve as a predictive tool for investment.
- **ROI Optimization:** The **CalculateROI** function revealed that while AAA games generate the highest gross revenue, certain AA and Indie titles powered by royalty-based engines (e.g., Unreal Engine) often yield a higher percentage return on investment due to lower development budgets.

3. Discussion and Conclusion

- **Centralization vs. Fragmentation:** The project successfully addresses the core problem of fragmented industry data. By centralizing disparate entities (DLCs, Platforms, Engines) into a single relational nexus, the system provides a holistic view of the gaming ecosystem that is otherwise impossible to achieve with manual tracking.
- **Scalability and Accountability:** The inclusion of atomic transactions (**RegisterGameComplete**) and audit-ready views ensures the system is scalable for large publishing houses managing dozens of concurrent projects. Every change to a platform listing or pricing strategy is tracked, fostering a culture of accountability.
- **Strategic Impact:** The primary outcome of this implementation is the shift from **reactive** to **proactive** management. Rather than simply recording sales, the system allows for real-time benchmarking against competitors, enabling publishers to adjust pricing and distribution strategies dynamically.

Table 1: Sample Developer Registry

This table shows the normalization of studio data, including independence status.

Developer ID	Name	Country	Founded	Team Size	Independent	Parent Company
101	Ironclad Studios	USA	2008	250	No	Nexus Interactive
102	Neon Dreams	Japan	2015	45	Yes	NULL

103	Blue Baltic	Estonia	2012	12	Yes	NULL
104	Titan Games	Canada	1999	600	No	Global Pub Corp

Table 2: Sample Game Intelligence (Core Schema)

This table demonstrates the tracking of financial metrics and metadata.

ID	Title	Release Date	Publisher	Price (USD)	Metacritic	Revenue (Est)
501	Cyber Odyssey	2023-11-10	Major Alpha	\$69.99	88	\$450,000,000
502	Pixel Quest	2024-01-20	Indie Label	\$14.99	75	\$1,200,000
503	Shadow Protocol	2023-05-15	Mid-Tier X	\$39.99	82	\$15,000,000
504	Neon Racer	2023-08-30	Self-Pub	\$0.00	68	\$500,000

Table 3: Genre Performance Analytics (View Output)

*This table represents the output of your **GenrePerformance** View, providing high-level market insights.*

Genre Name	Game Count	Avg. Revenue (USD)	Avg. Metacritic	Avg. User Review
RPG	42	\$120,500,000	84.5	88.2%
FPS	115	\$85,000,000	78.2	72.4%
Roguelike	28	\$4,200,000	81.0	91.5%

Simulation	15	\$18,900,000	74.3	76.8%
------------	----	--------------	------	-------

Table 4: Cross-Platform Commercial Listings

Illustrates how the system tracks localized pricing and exclusivity across different storefronts.

Listing ID	Game Title	Platform	Price	Exclusive?	Sub. Included?
1	Cyber Odyssey	PS5	\$69.99	Yes	No
2	Pixel Quest	Steam	\$14.99	No	No
3	Shadow Protocol	Xbox Series X	\$39.99	No	Yes (GamePass)
4	Pixel Quest	Switch	\$14.99	No	No

Concluding Remarks

The development of the Obsyria system represents a significant step forward in addressing the data management challenges prevalent in the modern gaming industry. By transitioning from fragmented, ad-hoc data tracking to a centralized, 3NF normalized relational framework, this project has successfully demonstrated how robust database architecture can transform raw information into strategic assets.

Throughout the implementation, the project achieved several key technical milestones:

- **Integrity and Automation:** The integration of database-level triggers ensures that financial and performance data remain consistent and valid, regardless of the scale of entry.

- **Actionable Analytics:** The use of complex stored procedures and business intelligence views has proven that a database can do more than just "store" data—it can actively classify, benchmark, and identify market opportunities in real-time.
- **Transactional Stability:** By utilizing atomic transactions, the system maintains a high degree of accountability and reliability, essential for managing high-stakes projects in a multi-platform environment.

In conclusion, Obsyria provides a scalable and efficient infrastructure for publishers and developers to navigate the complexities of the digital entertainment market. The system not only minimizes the risks associated with data redundancy and fragmentation but also empowers stakeholders with the analytical tools necessary for data-driven decision-making. Future iterations of this system could further leverage this normalized foundation to incorporate machine learning for predictive sales modeling or automated budget optimization, continuing the evolution of gaming intelligence.

Any societal relevance of your work

1. Promoting Transparency in the Gaming Industry

The global gaming market represents a multi-billion dollar economy, yet critical datasets regarding development budgets, monetization architectures, and publishing practices remain obscured from public view. NGI addresses this information asymmetry by architecting a structured, queryable framework that brings transparency to funding and distribution models. This infrastructure empowers consumers to make data-driven purchasing decisions while holding studios accountable for predatory practices, such as aggressive pay-to-win monetization.

2. Supporting Independent and Small Developers

For indie studios, failure is often a consequence of fragmented market intelligence rather than a lack of creative talent. The `GetGenreOpportunity` procedure directly mitigates this risk by analyzing profitability trends relative to specific team sizes and budget constraints. By democratizing access to high-level analytical insights previously reserved for major publishers, NGI provides small developers with the strategic foundation necessary to identify and occupy profitable market gaps.

3. Encouraging Ethical Monetization

By correlating `monetization_model` data with `user_review_pct`, the system establishes a measurable link between revenue strategies and player sentiment. The `DLCMonetization` view highlights the negative impact of aggressive expansion pricing on long-term user satisfaction, providing developers with empirical evidence to adopt fairer, player-centric business models.

4. Advancing Data Literacy in the Tech Community

This project serves as a functional demonstration of how relational database theory and SQL optimization can be applied to a high-impact consumer domain. By bridging the gap between academic normalization standards and practical industry requirements, NGI provides an accessible and relatable framework for students and tech professionals to enhance their data literacy and technical proficiency.

5. Contributing to Sustainable Business Practices

Through the `CalculateROI` function and the associated `GameROI` view, the system allows studios to rigorously evaluate the relationship between development expenditure and actual returns. This promotes more sustainable financial planning in an industry vulnerable to budget overruns, thereby protecting the livelihoods of thousands of developers and ensuring studio longevity.

6. Potential for Consumer Advocacy and Policy

A scaled implementation of this architecture could provide journalists, researchers, and policymakers with the tools necessary to regulate loot boxes, subscription lock-ins, and exclusivity agreements. By capturing critical dimensions such as `is_exclusive` and `subscription_included`, NGI lays the technical foundation for impactful data journalism and consumer advocacy within the digital landscape.

Future scope

The **Future Scope** of the **Obsyria** project involves transitioning from a descriptive management system to a predictive, real-time analytical ecosystem.

1. Integration of Machine Learning and AI

The most significant future expansion is the implementation of **Predictive Analytics**. By leveraging existing data through machine learning models, the system could predict a game's success (e.g., sales or Metacritic score) before launch, enabling publishers to optimize budgets and release strategies.

2. Real-Time Telemetry and Live-Ops Tracking

Future iterations may include **Live-Service Telemetry**, tracking metrics such as DAU, MAU, and player churn in real time. This would support dynamic decision-making for live-service games.

3. Social Sentiment and Community Analysis

Integration with social media platforms (Twitter, Reddit, Discord) would enable **Sentiment Analysis**, providing a real-time “Social Pulse” of player feedback and market reception.

4. Blockchain and Virtual Economy Monitoring

A future module could track **in-game asset value and transactions**, supporting secure management of virtual economies.

5. Global Cloud Scaling and API Expansion

Migration to a **distributed cloud architecture** (e.g., AWS RDS or Google Cloud) would enhance scalability and enable real-time global collaboration.

6. Personalized Recommendation Engines

By incorporating player behavior data, NGI could power **recommendation systems**, suggesting games based on genre and platform preferences.

Team details and individual contribution

- **Parth Malik — Database Design & Schema Architecture:** Spearheaded the conceptual modeling phase by architecting the complete ER diagram using Chen's notation to map complex entities and cardinalities. Translated the design into a production-grade relational schema across nine tables, ensuring full normalization to Third Normal Form (3NF) while enforcing rigorous referential integrity through primary keys, foreign keys, and domain-specific ENUM and CHECK constraints.
- **Abhishek Mishra — Dashboard, Database Connectivity & Verification:** Engineered the React-based frontend dashboard and established a seamless integration layer with the backend database. Managed the mapping of interactive UI components to live SQL queries, ensuring real-time accuracy of displayed industry

metrics. Conducted end-to-end system validation to verify operational stability from high-level user interactions down to low-level database response cycles.

- **Reia Baid — Cloud Deployment & Query Optimization:** Managed the deployment of the MySQL instance on Aiven Cloud, configuring SSL-encrypted connectivity and environment-variable isolation. Optimized system performance by implementing B-Tree indexes on high-traffic columns such as `title`, `release_date`, and `metacritic_score`. Validated performance gains through rigorous benchmarking to quantify latency reductions across production-scale datasets.
- **Advit Sharma — Backend Development, Testing & Documentation:** Architected the Node.js backend infrastructure, including the `src/db.js` connection pooling module and automated data seeding scripts. Developed and stress-tested the database business logic layer, encompassing five stored procedures, four triggers, and the `CalculateROI` scalar function. Compiled the comprehensive technical documentation suite, including the README, Viva Guide, and final strategic report.

References

- [1] Silberschatz, A., Korth, H. F., and Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.
- [2] Elmasri, R., and Navathe, S. B. (2017). *Fundamentals of Database Systems* (7th ed.). Pearson Education.
- [3] MySQL Documentation. (2024). *MySQL 8.0 Reference Manual — Stored Procedures, Triggers, and Views*. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6), 377–387.

[5] Newzoo. (2023). *Global Games Market Report 2023*. Newzoo BV.
<https://newzoo.com/resources/trend-reports/global-games-market-report>

Use of AI

The development of the **Obsyria** system involved the selective and transparent utilization of AI tools, primarily as a productivity aid for non-core project components, like the frontend and dashboard (layout, dark mode, and UI integration with the REST backend); it also helped ideate and optimize those interfaces and prototype UI flows. The core database was designed and implemented entirely by the team: ER diagrams, normalization to 3NF, stored procedures, triggers, functions, and atomic transactions were done manually. This implementation ensured that the system's core—the relational engine—was built entirely upon technical proficiency and the academic principles of DBMS optimization, leveraging AI only as a strategic asset for presentation rather than a substitute for technical understanding.

video

<https://drive.google.com/drive/folders/1DZmlYFBBelFqELPtq6kuPLoszcS-Nh1Z?usp=sharing>